Monte Carlo Forecasting for Scrum Teams

How to turn sprint history into delivery odds —
and why a single delivery date is a coin toss

THE WHOLE METHOD IN FOUR LINES

Write each past sprint's result on a slip of paper and put the slips in a bowl.
For every sprint left before your deadline, draw a slip, note it, and **put it back**.
Add up the slips: that's one possible future. Do it a thousand times.
The share of futures that reach your target is your probability.

A practical guide · free to share · Digital2DNA

Contents

1. The problem with a single number
2. The idea: a bowl of sprints
3. What you need to start
4. Step 1 — Your history
5. Step 2 — Your goal
6. Step 3 — One replay
7. Step 4 — A thousand replays
8. Why not a bell curve?
9. Two measures: effort and scope
10. How to commit
11. What this cannot do
12. A worked example, end to end
13. Build it yourself in a spreadsheet
14. Common questions
15. Cheat sheet and glossary

1. The problem with a single number

Here is how most teams forecast. Velocity is about 40 points a sprint. There are 8 sprints before the deadline. So we'll deliver 320 points, and the 300 points of scope will be comfortably done.

That arithmetic is not wrong, but the conclusion is. 320 is the *middle* of what might happen. Roughly half of all possible futures land below it. Presenting it as the plan is presenting a coin toss as a commitment — and when the team misses, it looks like a failure of execution rather than what it was: a forecast with no margin in it.

The missing information is not the average. It is the **spread**. A team that delivers 39, 41, 40, 40, 39 is a very different proposition from one delivering 12, 58, 33, 47, 50, even though both average 40. The first can commit to nearly its average. The second cannot.

Monte Carlo forecasting keeps both the average and the spread, and answers the questions a stakeholder actually has:

- **What are the odds we hit this date?**
- **What can we commit to and be fairly sure of?**
- **How much margin do we actually have?**

> **EXPLAINING IT TO A STAKEHOLDER**
> "We can give you a date with a 50% chance, or a date with an 85% chance. They're different dates. Which would you like to plan around?"

2. The idea: a bowl of sprints

A Monte Carlo simulation answers "what are the odds?" by playing the situation out many times with realistic random variation, then counting how often each outcome happened. It was named after the casino. There is nothing exotic in it.

For sprint forecasting the mechanic is this:

1. Write what each past sprint delivered on its own slip of paper.
2. Put every slip in a bowl.
3. You have 8 sprints before the deadline. Draw a slip, write the number down, **put the slip back**, and repeat until you have 8 numbers.
4. Add them up. That is one plausible future.
5. Tip the bowl out and do the whole thing again. A thousand times.

Now you have a thousand futures. Count how many reached your target: that is your probability. Sort them and look at the one 15% from the bottom: that is the number you'd beat 85 times in 100.

> **WHY THE SLIP GOES BACK IN THE BOWL**
> "With replacement" is the part people skip, and it matters. If you drew without replacing, you would just be reshuffling the same sprints — and a replay the length of your history would always total the same thing. Putting the slip back makes each draw independent, so a replay *can* contain two disastrous sprints in a row even though your history only held one. Real projects do that. Your forecast should be allowed to.

Notice what the bowl method does *not* require. You do not have to decide what shape your velocity data has, or whether it is "normal", or fit any curve to it. The bowl only ever contains sprints that really happened, so whatever pattern your team has — steady, lumpy, occasionally catastrophic — is already in there. Statisticians call this **resampling** or **bootstrapping**. It is the most assumption-free tool in the box.

3. What you need to start

Input	Notes
Sprint history	For each past sprint: story points completed, and stories completed. Five sprints is a workable minimum; eight to twelve is comfortable. Only count work that actually met your definition of done.
How far back to look	Older sprints may describe a team that no longer exists. Drop sprints from before a real change (new members, a different kind of work, a new definition of done).
Start and target dates	The window you are forecasting.
Sprint length	Must match the sprints in your history. This is the single most common input error — see the warning below.
Target scope	In story points. A story count can be derived from it.

SPRINT LENGTH IS NOT A SPEED DIAL

If your history is made of 2-week sprints and you tell the tool sprints are 1 week, it will fit twice as many sprints before the deadline and roughly double the forecast. The team did not get faster; the inputs contradicted each other. Sprint length only answers "how many of those past-looking sprints fit before the date?"

4. Step 1 — Your history

This is the whole input to the model. Everything else is arithmetic.

Two summary numbers are worth computing, not because the forecast needs them — it draws from the raw sprints — but because they tell you at a glance what kind of team you are looking at:

```
average   μ = sum of sprints / number of sprints
spread    σ = √( sum of (sprint − μ)² / (n − 1) )     [a spreadsheet's STDEV]
```

Read σ as "a typical sprint lands this far from the average". A team averaging 40 with σ = 3.6 is predictable. The same average with σ = 15 is not, and no amount of forecasting will make it so — though the forecast will at least tell you the truth about it.

The `n − 1` rather than `n` corrects for the fact that you are estimating spread from a sample rather than measuring a whole population. Every spreadsheet's `STDEV` does this.

How much history?

Sprints	What you can expect
1–2	Not a forecast. The bowl holds one or two slips; every replay is nearly identical.
3–4	Directional at best. The shape of the team's variation is barely represented.
5–7	Usable. Treat the tails (P95) with suspicion.
8–12	Comfortable. Enough slips for the bowl to represent real variety.
12+	Good — but now ask whether the oldest ones still describe this team.

5. Step 2 — Your goal

```
sprints available   n  = ⌊ (target date − start date) / sprint length ⌋
story size          s  = total points in history / total stories in history
target stories      Tₛ = ⌊ target points / s ⌋
```

Both roundings go *down*, deliberately. A sprint that ends after the deadline cannot help you, and a fraction of a story is not a delivered story.

Deriving a story target from the point target lets you check the forecast two ways — see section 9. If your coming work will be sliced differently from your past work, set the story size by hand instead.

6. Step 3 — One replay

One replay is one possible future. To build it, draw `n` slips from the bowl with replacement and add them up.

```
for each of the n remaining sprints:
    pick a past sprint at random (any one, every time, equally likely)
    add its result to the running total
the total is one replay
```

That is the entire mechanic. Two properties follow from it, and both are features:

- **A replay can never contain a sprint the team has never had.** If your worst sprint was 12 points, no future in the simulation contains a 9-point sprint.
- **The shape of your data survives.** If one sprint in ten is a disaster, roughly one draw in ten is a disaster, and replays with two or three disasters appear at exactly the rate chance allows.

7. Step 4 — A thousand replays

Repeat step 3 a thousand times and sort the totals. Every number the method reports is read off that sorted pile.

The probability

```
P(hit target) = how many replays reached the target / how many replays
```

The percentiles

"P85" means **the amount you reach in 85% of replays**. That is the value with 85% of the pile *above* it — which makes it the 15th percentile of the sorted pile, not the 85th. This trips people up constantly.

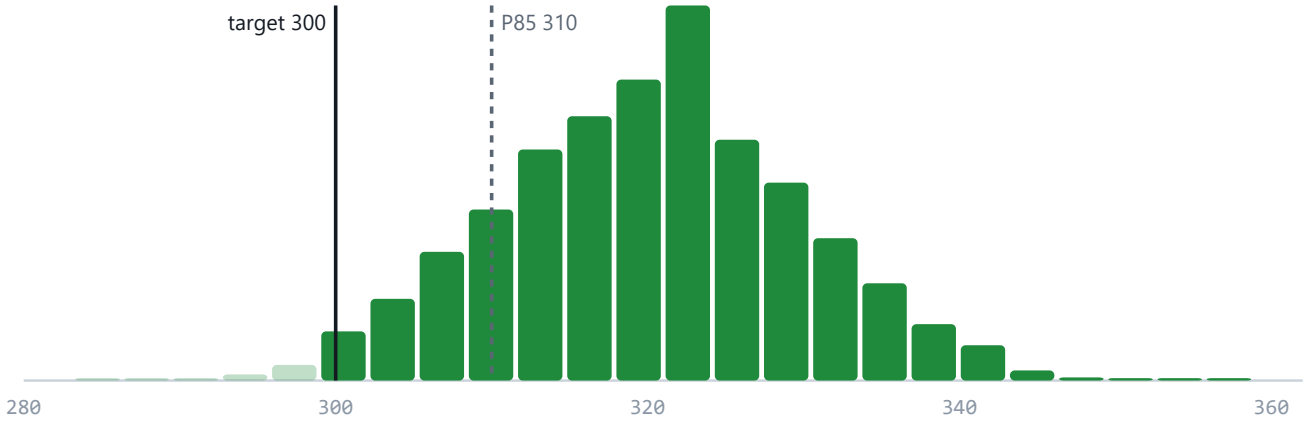
Label	Plain meaning	Where in the sorted pile	Use it for
P50	a coin toss	50th percentile	Nothing you'd commit to. This is what velocity × sprints gives you.
P75	probable	25th percentile	Internal planning, stretch goals.
P85	safe commitment	15th percentile	What you tell stakeholders. Wrong about one time in seven.
P95	near certain	5th percentile	Contractual dates, regulatory deadlines.

EXPLAINING P85

"This is the number we'd hit in 85 out of 100 futures. Committing here means we're wrong about one time in seven. P50 — which is what multiplying velocity by sprints gives you — is wrong half the time."

The histogram

Bucket the pile and draw it, and the result is the most useful picture in agile forecasting: a visible distribution of outcomes with your target marked on it. The share of the bars to the right of the target line *is* your probability.



A thousand replays for the example in section 12. Solid line: the 300-point target. Dashed: P85. Pale bars are futures that fell short — about 1.5% of them.

Why the numbers move when you run it again

Every run draws new slips, so the percentage wobbles. A proportion estimated from N replays carries a standard error of `√(p(1−p)/N)` — about ±0.5 points at 1,000 replays, ±0.16 at 10,000. The underlying answer does not change; only the sampling noise does. Treat the last digit as decoration.

More replays buy precision in the *estimate*. They buy nothing in accuracy — that comes from the history. A thousand replays off five sprints of history is a precise answer to a badly-informed question.

8. Why not a bell curve?

Many Monte Carlo spreadsheets — including an earlier version of our own — skip the bowl and instead fit a normal distribution (a bell curve) to velocity, then draw from that. It is tempting: a bell is fully described by just the average and the spread, and every spreadsheet has a function for it. It is also wrong in ways that matter.

Velocity is not bell-shaped

- **A bell curve is symmetric. Velocity is not.** A team can have a catastrophic sprint and deliver almost nothing. There is no equal-and-opposite sprint where they deliver double. The real distribution leans, with a longer tail on one side.
- **A bell curve extends forever in both directions.** It assigns real probability to negative velocity, which is meaningless. Implementations usually patch this by clamping at zero — which is a sign the model is wrong, not a fix.
- **A bell curve understates clustered bad luck.** Teams have genuinely bad sprints more often than a bell predicts: incidents, outages, key people away. Those are the futures you most need represented.

What it costs you in practice

Here is a real-shaped team: fifteen sprints, mostly in the low twenties, with two bad ones (3 and 5 points) when an incident swallowed the sprint.

```
22 24 21 23 25 20 24 22 23 21 3 24 22 5 23      average 20.1, spread 6.7
```

Horizon	Target	Bell curve says	Bowl says	Gap
4 sprints	76	63%	59%	4 points optimistic
6 sprints	114	66%	76%	10 points pessimistic
10 sprints	190	70%	72%	2 points

The errors do not even point the same way, which is what makes them dangerous — you cannot correct for them with a rule of thumb. Ten percentage points is the difference between "commit" and "don't".

The bell also produces outright impossible futures. For this team over 6 sprints, the best conceivable outcome is 6 × 25 = 150 points. The bell curve happily generates totals above 172.

"But doesn't the Central Limit Theorem rescue it?"

Partly, and it is worth knowing the argument because someone will raise it. The Central Limit Theorem says that the *sum* of many independent draws tends toward a bell shape whatever the individual draws look like. So over a long horizon, modelling the *total* as normal is not indefensible — and indeed in the table above the gap narrows at 10 sprints.

But: it is a defence of the output, not of the input. It weakens exactly when you need the forecast most — short horizons, few sprints, lumpy teams. And it requires you to explain the Central Limit Theorem to defend your tool. The bowl requires no defence at all. Nobody has ever needed a statistics lecture to believe that drawing past sprints out of a bowl is a fair way to imagine future ones.

THE RULE

Don't assume a distribution when you have the data. If you have the sprints, use the sprints.

A lognormal distribution is sometimes suggested as a better fit than a bell, since it is non-negative and asymmetric. It is better. But it is still an assumption about shape, and you do not need one.

9. Two measures: effort and scope

Story points measure **effort**. Stories completed measure **scope** — how many things actually got finished. Forecasting both, from the same bowl, costs nothing and catches problems a single measure hides.

Draw whole sprints, not separate numbers

When you draw a slip, take the sprint's points *and* its stories together, as they actually occurred. They are correlated in real data — typically strongly — and keeping them paired preserves that. Drawing them independently would quietly imply a team could have a huge-points, tiny-stories sprint as easily as not.

This also makes a genuinely useful number available: **the odds that both targets land**. Since a project is not done until the effort *and* the scope are delivered, that is the honest headline. It is always lower than either measure alone — and higher than multiplying the two probabilities together, because the measures rise and fall as one.

Reading a disagreement

Pattern	What it usually means	What to do
Point odds good, story odds poor	Effort is fine; *finishing* is erratic. The number of stories closed per sprint swings more than the points do.	Keep stories small and similar in size; limit work in progress; cut per-story overhead (reviews, hand-offs, deploys).
Story odds good, point odds poor	A steady number of stories closes, but effort per sprint swings — a few large stories carry most of the points.	Split the big ones.
Both similar	Effort and scope agree. The forecast isn't hiding a sizing problem.	Commit on the numbers.

A GAP IS ABOUT CONSISTENCY, NOT SIZE

You will see advice saying that high point odds with low story odds means stories are "too small" and should be merged. Be careful with that: it contradicts the point of small stories, which is short feedback loops. A gap between the two odds is a *variability* signal — it says one measure is more erratic sprint to sprint, which usually means inconsistent sizing, not uniformly small stories.

10. How to commit

1. **Pick your confidence level before you look at the answer.** P85 for a normal commitment, P95 if the date is contractual. Choosing afterwards is how forecasts become wishful.
2. **Commit on whichever measure is tighter.** Compare each measure's P85 against its own target; the lower ratio is your real constraint. You need both to land.
3. **Quote a range, not a point.** "P50 is 320, P85 is 310" tells the whole story in nine words.
4. **Re-run every sprint.** A forecast is a snapshot. The most valuable thing about it is watching it move.
5. **Say the caveats out loud.** The model knows nothing about the holiday season, the rewrite, or the person leaving. You do.

A COMPLETE, HONEST FORECAST, SPOKEN

"Based on our last ten sprints, there's an 85% chance we deliver at least 310 points by 24 June, and a 50% chance we get past 320. The 300 we've scoped sits comfortably inside that. The main thing that would change it is the two weeks of holiday in May, which isn't in these numbers."

11. What this cannot do

Every forecasting method has a boundary. Knowing where yours is, and saying so, is what separates a forecast from a promise.

Assumption	What it means	What to do about it
The future resembles the past	Same team, same kind of work, same definition of done.	The big one. Trim the history when something real changed, and name known future disruptions as caveats.
Sprints are independent	A bad sprint doesn't make the next one bad.	Not quite true — problems cluster. This makes the forecast slightly optimistic. Longer history captures more clusters.
Your worst sprint is as bad as it gets	Resampling can't produce an outcome worse than the worst in the bowl.	The real limitation of the bowl method. With little history, treat P95 as optimistic.
Scope is fixed	Nothing gets added.	Not modelled at all. Re-run when scope changes; keep old forecasts to show the drift.
Points mean the same thing over time	A 5 today is a 5 from last quarter.	Re-estimation and point inflation break this. Story counts are more robust — another reason to track both.
Capacity is baked in	Holidays and on-call are only represented if they were in the history.	Either accept the average absorbs them, or restrict history to comparable sprints.

THE HONEST SUMMARY

This does not predict the future. It tells you what your own history implies, if nothing changes. Everything you know that the history doesn't is a caveat you add out loud.

12. A worked example, end to end

A team with eight sprints behind it, eight sprints before the deadline, and 300 points of scope.

Sprint	1	2	3	4	5	6	7	8	Total
Points	42	36	45	38	41	39	44	35	320
Stories	9	7	10	8	8	9	8	7	66

The inputs

```
points   average 40.0   spread 3.6
stories  average 8.25   spread 1.0
story size      320 / 66 = 4.85 points per story
target stories  ⌊300 / 4.85⌋ = 61
sprints left    ⌊112 days / 14⌋ = 8
```

One replay, by hand

Eight draws from the bowl might give: 38, 45, 35, 41, 38, 44, 36, 45. Note that 38 and 45 each came up twice — that is replacement working. Total: **322 points**. One future, and it clears 300.

Another might give 35, 36, 38, 35, 39, 36, 41, 35 = **295**. That one misses. Do this a thousand times and count.

The result

Measure	Odds of hitting target	P50	P75	P85	P95
Points (target 300)	98.5%	320	313	310	304
Stories (target 61)	98.2%	66	64	63	62
Both targets	**97.4%**	Not 98.5% × 98.2% = 96.7% — the measures move together, so the true figure is higher.			

What to say

"There's a 97% chance we deliver both the 300 points and the 61 stories by the deadline. At 85% confidence we'd commit to 310 points, which is 10 clear of the scope. This is a predictable team — the worst replay in a thousand still delivered 280."

The range of totals this bowl can produce is 8 × 35 = 280 at worst and 8 × 45 = 360 at best. Every forecast number sits inside that, by construction.

13. Build it yourself in a spreadsheet

You do not need software for this. Any spreadsheet does it in a few formulas.

Put your sprint history in `A2:A13`. Then one replay of `n` sprints — summing `n` random draws with replacement — is:

```
=SUMPRODUCT(INDEX($A$2:$A$13, RANDARRAY(n, 1, 1, COUNT($A$2:$A$13), TRUE)))
```

Fill that down a thousand rows into column C. Then:

```
probability   =COUNTIF(C:C, ">=" & target) / COUNT(C:C)
P50           =PERCENTILE(C:C, 0.50)
P75           =PERCENTILE(C:C, 0.25)
P85           =PERCENTILE(C:C, 0.15)
P95           =PERCENTILE(C:C, 0.05)
sprints left  =MAX(0, INT((target_date - start_date) / sprint_days))
target scope  =INT(target_points / (SUM(points) / SUM(stories)))
```

Notes. `RANDARRAY(...,TRUE)` gives whole numbers and needs a modern Excel or Google Sheets. The sheet re-rolls on every edit; that is the simulation re-running, not a bug. To forecast stories from the same draws, generate the row indices once into their own column and use `INDEX` twice against it — that keeps points and stories paired.

If your spreadsheet lacks `RANDARRAY`, the older equivalent per cell is `=INDEX($A$2:$A$13, RANDBETWEEN(1, COUNT($A$2:$A$13)))` — put `n` of those across a row and sum them, then fill down a thousand rows.

14. Common questions

Isn't this just velocity times sprints with extra steps?
Velocity × sprints is one of the thousand futures — the middle one. The method keeps the other 999, which is where the odds and the margin come from.

Where does the randomness come from?
Entirely from your own history. The bowl contains nothing invented.

Why 1,000 replays? Why not a million?
1,000 puts the noise on a percentage at roughly half a point — below anything you'd act on. 10,000 smooths the histogram. A million changes nothing you can see.

Why do the numbers change every time I run it?
New draws. The underlying answer is fixed by your history; only sampling noise moves. If 97% becomes 96%, nothing happened.

Why is P85 the 15th percentile and not the 85th?
Because "85% confidence" means 85% of futures do at least this well — so 85% are above it and 15% below.

Can I use this with story counts instead of points?
Yes, and many teams prefer it. Story throughput needs no estimation at all, which sidesteps point inflation entirely. The method is identical — put story counts in the bowl.

What about Kanban, where there are no sprints?
Same method with a different bucket: use throughput per week (or per day) and count weeks instead of sprints. Nothing else changes.

How do I handle a team that just changed size?
Drop the history from before the change and accept a thinner bowl, or keep it and say the forecast is based on a different team. Do not average across the change and pretend it's one team.

What if we have only three sprints of history?
You can run it, and it will produce numbers, but the bowl holds three slips — the range of futures it can imagine is tiny and the tails are meaningless. Present it as directional and re-run as history accumulates.

Does this handle scope creep?
No. The target is fixed input. Re-run when scope changes — and keep the old forecasts, because the trend across them is often the most revealing thing you have.

Our velocity is trending up. Doesn't resampling miss that?
Yes — it treats all history as equally representative. If there's a genuine trend, use only the recent sprints that reflect the new level. Be honest about whether it's a trend or a run of good luck; three good sprints is usually the latter.

Can it tell us a date rather than an amount?

Yes, by running it the other way: draw sprints until the remaining scope is consumed, and record how many sprints it took. A thousand runs gives you a distribution of dates. Same bowl, same principle.

Isn't 85% arbitrary?

It's a convention, not a law. It happens to sit where most stakeholders' intuition about "safe" lands — wrong roughly one time in seven. Use P95 when being wrong is expensive.

Can it be wrong?

Confidently, if the future stops resembling the past. It's not a prediction; it's an implication. That distinction is the forecaster's job to communicate.

15. Cheat sheet and glossary

The method

```
1. one slip per past sprint
2. draw n slips WITH replacement
3. sum them = one replay
4. repeat 1,000 times
5. count and sort
```

Reading the pile

```
P(hit)  = replays ≥ target / all
P50     = 50th percentile
P75     = 25th percentile
P85     = 15th percentile
P95     =  5th percentile
```

Sizing the goal

```
n  = ⌊days / sprint length⌋
s  = Σpoints / Σstories
Tₛ = ⌊target points / s⌋
```

Noise on a percentage

```
± √( p(1−p) / N )
p=.97, N=1,000  → ±0.5 pts
p=.97, N=10,000 → ±0.16 pts
```

Confidence to use

```
P50  never commit here
P75  internal stretch
P85  normal commitment
P95  contractual dates
```

Committing

```
pick the level FIRST
commit on the tighter measure
quote a range
re-run every sprint
say the caveats out loud
```

Glossary

Monte Carlo simulation	Answering "what are the odds?" by running many random trials and counting outcomes.
Resampling / bootstrapping	Drawing randomly from your actual observations instead of from a fitted curve. The bowl.
With replacement	Putting each drawn slip back, so it can be drawn again. Makes draws independent.
Velocity	Story points completed per sprint.
Throughput / flow	Items completed per sprint (or per week).
Average (mean)	The middle of your history. Useful summary; not a commitment.
Standard deviation	How far a typical sprint sits from the average. The "±".
Distribution	The shape of all possible outcomes and how likely each is.
Normal distribution	The bell curve. Symmetric, unbounded — and a poor fit for velocity.
Percentile	The value below which a given share of outcomes falls.
P85	The amount reached in 85% of replays — the 15th percentile of the pile.
Central Limit Theorem	Sums of many independent draws tend toward a bell shape, whatever the individual draws look like.

On the method. Resampling for agile forecasting is long-established practice; Troy Magennis and Daniel Vacanti have both written extensively on it, and it underpins most serious forecasting tools. Our thanks to **Mike Cohn**, whose direct feedback prompted us to replace a normal-distribution model with resampling — and whose bowl-of-paper-slips explanation is the one reproduced here.

Using this guide. Free to share and use, including inside your own organisation. If it's useful, a link back is appreciated.

Digital2DNA — we deliver working software for organizations where complexity is the reality and progress can't wait.
`digital2dna.com`